Lab 03. Thiết kế, huấn luyện và kiểm thử Tiny CNN trên Edge Impulse

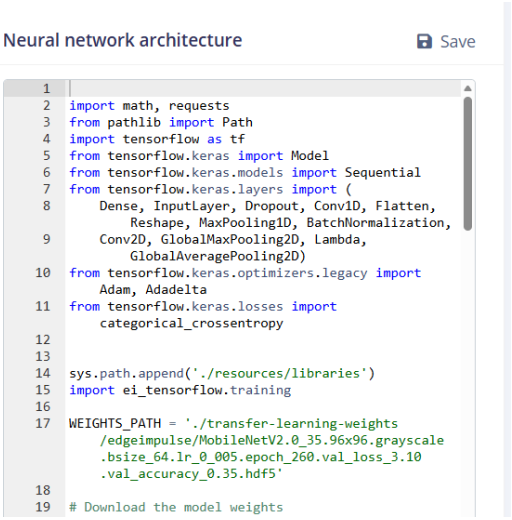
Step 1. Mở dự án và kiểm tra dữ liệu huấn luyện

Mở Edge Impulse Studio, truy cập dự án Brain_MRI_PYNQ_Z2. Dữ liệu đầu vào đã được chuẩn bị trong Lab 02: 4.955 ảnh Training và 995 ảnh Test thuộc bốn lớp glioma, meningioma, no_tumor và pituitary. Không thay đổi phân chia Train/Test đã nhập.

Trong Impulse design, mở Learning Block Transfer Learning (Images). Mục đích là tự thiết kế mạng CNN thay cho MobileNetV2 mặc định, sau đó xuất trọng số Conv2D phục vụ nghiên cứu tăng tốc trên PYNQ-Z2.

Step 2. Chuyển sang Keras Expert Mode

Trong Neural Network settings, nhấn biểu tượng ba chấm và chọn Switch to Keras (expert) mode. Giao diện mới hiển thị tập lệnh Python dùng TensorFlow/Keras. Không sử dụng trọng số MobileNetV2 trong mô hình tự thiết kế.



Hình 3.1. Giao diện Keras Expert Mode trên Edge Impulse.

Step 3. Xây dựng mô hình CNN thử nghiệm lần thứ nhất

Dùng ảnh 64 × 64 × 1 (Grayscale), hai lớp Conv2D lần lượt có 4 và 8 filters, kernel 3 × 3, sau mỗi lớp là ReLU và MaxPooling2D; cuối mạng dùng GlobalAveragePooling2D và Dense Softmax bốn lớp. Lớp Conv1 được tách riêng ReLU để thuận lợi đối chiếu tính toán phần cứng.

```
model = Sequential([
  InputLayer(input_shape=(64, 64, 1), name='x_input'),
  Conv2D(4, (3, 3), activation=None, name='conv1'),
  ReLU(name='relu1'), MaxPooling2D((2, 2)),
  Conv2D(8, (3, 3), activation=None, name='conv2'),
  ReLU(name='relu2'), MaxPooling2D((2, 2)),
```

```
    GlobalAveragePooling2D(),
    Dense(classes, activation='softmax')
])
```

Huấn luyện 20 Epoch, Batch Size 32, Learning Rate 0,0005. Kết quả Validation Accuracy INT8 đạt 51,1%, Loss 1,15. Do kết quả của cả Training và Validation đều thấp, mô hình ban đầu chưa đủ khả năng biểu diễn cho bộ ảnh MRI này.

Step 4. Cải tiến Tiny CNN lần thứ hai

Giữ Conv1 ở 4 filters, tăng Conv2 lên 16 filters, bổ sung Conv3 có 32 filters. Thay GlobalAveragePooling2D bằng Flatten, Dense 32 và Dropout 0,2. Huấn luyện 40 Epoch, Batch Size 32, Learning Rate 0,0005 bằng Adam.

Tại Epoch 40, Training Accuracy là 95,69%, Validation Accuracy là 91,12%, Training Loss là 0,1172 và Validation Loss là 0,2578. Trên giao diện kết quả lượng tử hóa INT8, Validation Accuracy đạt 90,8%, Validation Loss 0,35, Weighted F1-score 0,91.

Step 5. Thử nghiệm V3 và nhận biết Overfitting

Thử mở rộng CNN bằng lớp Conv4 có 64 filters, sử dụng Batch Normalization, Dropout 0,3 và các callback ReduceLROnPlateau, EarlyStopping. Trong log, Training Accuracy đạt cao nhất 99,75%, nhưng Validation Accuracy cao nhất chỉ 92,03%; checkpoint được khôi phục tại Epoch 12 theo Validation Loss.

Kết quả Validation INT8 cuối cùng là 90,9%, Loss 0,24. Việc tăng năng lực mạng có thể khiến mô hình ghi nhớ dữ liệu Training tốt hơn nhưng không tự bảo đảm tổng quát hóa. Không dùng Training Accuracy 99,75% làm kết quả đánh giá chung của đề tài.

Step 6. Cấu hình GPU và Advanced Training Settings

Ở Training processor, chọn GPU. Trong Advanced training settings, giữ Validation set size = 20%, Split train/validation set on metadata key để trống và bật Profile int8 model. Log của lần huấn luyện V4 xác nhận sử dụng NVIDIA Tesla T4.

Thiết lập	Giá trị sử dụng
Training processor	GPU (Tesla T4 theo log V4)
Validation set size	20% của tập Training
Split on metadata key	Để trống
Profile int8 model	Đã bật

Với 4.955 ảnh thuộc tập Training, Edge Impulse dùng 3.964 ảnh để cập nhật trọng số và 991 ảnh Validation. Tập Test 995 ảnh không được dùng khi huấn luyện hoặc lựa chọn siêu tham số.

Step 7. Nâng độ phân giải và huấn luyện Tiny CNN V4

Mở Impulse design → Create impulse, đổi Image width và Image height từ 64 lên 96, giữ Fit longest axis và 100% dữ liệu; sau đó lưu, xác nhận Color depth = Grayscale và chạy lại Generate Features. Việc chạy lại là bắt buộc khi thay đổi kích thước đặc trưng đầu vào.

V4 sử dụng các lớp Conv2D 8, 16, 32, 64 filters; Batch Normalization, Flatten, Dense 64, Dropout 0,4; 70 Epoch tối đa, Learning Rate 0,0003. Log ghi nhận 127.556 tham số và khôi phục checkpoint có Validation Loss thấp nhất tại Epoch 62. Giao diện INT8 báo Accuracy 95,7%, Loss 0,12.

Step 8. Thiết kế phiên bản Tiny CNN V5

Đầu vào 96 × 96 × 1 giữ nguyên. Kiến trúc V5 tăng Conv2 lên 24 filters, Conv3 lên 48 filters, giữ Conv1 = 8 và Conv4 = 64 filters. Thêm xoay và dịch chuyển ảnh nhẹ trong giai đoạn huấn luyện, Dropout 0,35, Learning Rate 0,0003; tối đa 80 Epoch.

Thành phần	Cấu hình V5
Input	96 × 96 × 1, Grayscale
Conv1 - mục tiêu FPGA	8 filters; 3 × 3; valid; stride 1; ReLU tách riêng
Conv2, Conv3, Conv4	24, 48, 64 filters; 3 × 3; same
Classifier	Flatten → Dense 64 → Dropout 0,35 → Softmax 4
Training	Adam; batch 32; LR 0,0003; tối đa 80 Epoch
Callbacks	ReduceLROnPlateau và EarlyStopping

Step 9. Nhập mã nguồn hoàn chỉnh Tiny CNN V5

Trong Keras Expert Mode, thay toàn bộ code cũ bằng các đoạn mã liền nhau dưới đây theo đúng thứ tự. Các biến args, train_dataset, validation_dataset, callbacks và BatchLoggerCallback do môi trường huấn luyện Edge Impulse cung cấp. Khi sao chép vào trình soạn thảo, giữ nguyên thụt lề Python.

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import (
    InputLayer, Conv2D, ReLU, MaxPooling2D,
    BatchNormalization, Flatten, Dense, Dropout
)

# 1. TRAINING CONFIGURATION
INPUT_SHAPE = (96, 96, 1)
BATCH_SIZE = 32
EPOCHS = 80
LEARNING_RATE = 0.0003
```

```
SEED = 42
ENSURE_DETERMINISM = args.ensure_determinism
tf.random.set_seed(SEED)
```

```
# 2. DATA AUGMENTATION
augmentation = tf.keras.Sequential([
    tf.keras.layers.RandomRotation(
        factor=0.012, fill_mode='constant'
    ),
    tf.keras.layers.RandomTranslation(
        height_factor=0.03,
        width_factor=0.03,
        fill_mode='constant'
    )
])

def augment_batch(images, labels):
    images = tf.cast(images, tf.float32)
    images = augmentation(images, training=True)
    images = tf.image.random_brightness(
        images, max_delta=0.02
    )
    images = tf.image.random_contrast(
        images, lower=0.95, upper=1.05
    )
    images = tf.clip_by_value(images, 0.0, 1.0)
    return images, labels
```

```
if not ENSURE_DETERMINISM:
    train_dataset = train_dataset.shuffle(
        buffer_size=512, seed=SEED,
        reshuffle_each_iteration=True
    )

train_dataset = train_dataset.batch(
    BATCH_SIZE, drop_remainder=False
)
if not ENSURE_DETERMINISM:
    train_dataset = train_dataset.map(
        augment_batch,
        num_parallel_calls=tf.data.AUTOTUNE
    )

prefetch_policy = (
    1 if ENSURE_DETERMINISM
    else tf.data.AUTOTUNE
)
train_dataset = train_dataset.prefetch(prefetch_policy)
validation_dataset = validation_dataset.batch(
    BATCH_SIZE, drop_remainder=False
).prefetch(prefetch_policy)
```

```
# 3. TINY CNN V5 ARCHITECTURE
regularizer = tf.keras.regularizers.l2(0.0001)

model = Sequential([
    InputLayer(input_shape=INPUT_SHAPE, name='x_input'),

    # Conv1: intended FPGA acceleration target
    Conv2D(8, (3, 3), strides=(1, 1),
           padding='valid', activation=None,
           name='conv1'),
    ReLU(name='relu1'),
    MaxPooling2D((2, 2), name='pool1'),
```

```
    Conv2D(24, (3, 3), padding='same',
           activation=None,
           kernel_regularizer=regularizer,
           name='conv2'),
    BatchNormalization(name='bn2'),
    ReLU(name='relu2'),
    MaxPooling2D((2, 2), name='pool2'),
```

```
    Conv2D(48, (3, 3), padding='same',
           activation=None,
           kernel_regularizer=regularizer,
           name='conv3'),
    BatchNormalization(name='bn3'),
    ReLU(name='relu3'),
    MaxPooling2D((2, 2), name='pool3'),

    Conv2D(64, (3, 3), padding='same',
           activation=None,
           kernel_regularizer=regularizer,
           name='conv4'),
    BatchNormalization(name='bn4'),
    ReLU(name='relu4'),
    MaxPooling2D((2, 2), name='pool4'),

    Flatten(name='flatten'),
    Dense(64, activation='relu',
          kernel_regularizer=regularizer,
          name='dense1'),
    Dropout(0.35, name='dropout1'),
    Dense(classes, activation='softmax',
          name='predictions')
])
```

```
# 4. TRAINING CALLBACKS
callbacks.append(
    BatchLoggerCallback(
        BATCH_SIZE, train_sample_count,
        epochs=EPOCHS,
        ensure_determinism=ENSURE_DETERMINISM
    )
)

callbacks.append(
    tf.keras.callbacks.ReduceLROnPlateau(
        monitor='val_loss', factor=0.5,
        patience=5, min_lr=0.00001,
        verbose=1
    )
)

callbacks.append(
    tf.keras.callbacks.EarlyStopping(
        monitor='val_loss', patience=16,
        restore_best_weights=True,
        verbose=1
    )
)
```

```
# 5. COMPILE AND TRAIN
model.compile(
    optimizer=tf.keras.optimizers.Adam(
        learning_rate=LEARNING_RATE
    ),
```

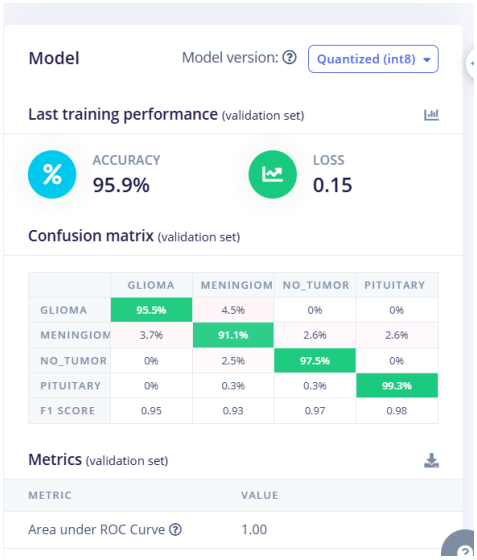
```
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
model.summary()

history = model.fit(
    train_dataset,
    validation_data=validation_dataset,
    epochs=EPOCHS,
    callbacks=callbacks,
    verbose=2
)
print('Tiny CNN V5 training completed.')
```

Step 10. Lưu mã và huấn luyện V5

Nhấn Save trong Neural Network Architecture, giữ Training processor = GPU, Validation set size = 20% và Profile int8 model = bật, sau đó nhấn Save & train. Kiểm tra Training output tới khi xuất hiện Job completed (success).

Kết quả thực nghiệm V5 trên tập Validation nội bộ: Accuracy INT8 = 95,9%, Loss = 0,15, Weighted F1-score xấp xỉ 0,96 theo màn hình kết quả. Việc tăng Accuracy so với V4 chỉ khoảng 0,2 điểm phần trăm và Loss cao hơn, do đó không khẳng định V5 vượt trội về mọi chỉ tiêu.



Hình 3.2. Mô hình V5 hoàn thành huấn luyện và kết quả Validation INT8.

Step 11. So sánh kết quả các phiên bản V1-V5

Phiên bản	Input	Validation Accuracy INT8	Validation Loss INT8
V1	64 × 64 × 1	51,1%	1,15
V2	64 × 64 × 1	90,8%	0,35
V3	64 × 64 × 1	90,9%	0,24
V4	96 × 96 × 1	95,7%	0,12
V5	96 × 96 × 1	95,9%	0,15

Phiên bản	Input	Validation Accuracy INT8	Validation Loss INT8

Validation Accuracy nêu trong bảng là kết quả Edge Impulse của phiên bản Quantized (int8). Kết quả này khác với Accuracy của Epoch cuối và không thay thế Test Accuracy. Bài toán bốn lớp không đạt mục tiêu 99% tổng thể trong các lần thử nghiệm đã thực hiện.

Step 12. Đánh giá mô hình trên 995 ảnh Test



Hình 3.3. Test Accuracy INT8, chỉ số đánh giá và Confusion Matrix.

Trong Edge Impulse, chọn Model testing → Classify all. Ở lần chạy đầu, tác vụ bị dừng với thông báo Evicted/Job failed; lần chạy lại thành công và hiển thị Job completed (success). Đây là lỗi tác vụ kiểm thử, không phải kết quả phân loại âm tính của mô hình.

Trên Test, V5 Float32 đạt Accuracy 92,96%, Weighted F1-score 0,94. Chuyển Model version sang Quantized (int8), Test Accuracy đạt 93,07%, Weighted Precision/Recall/F1-score đều xấp xỉ 0,94 theo giao diện.

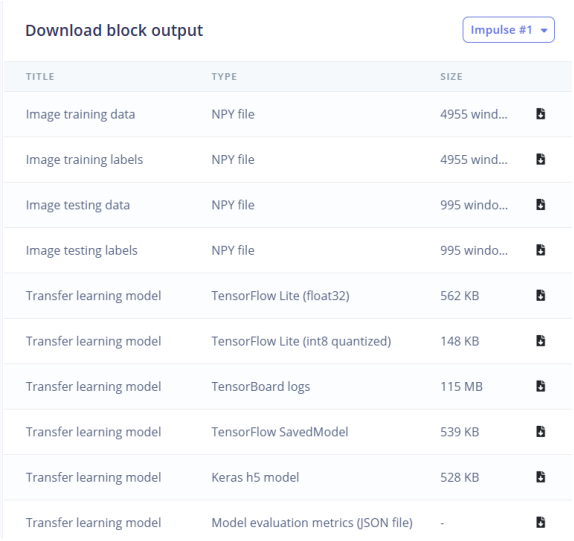
Chỉ số	Float32	INT8
Test Accuracy	92,96%	93,07%
Weighted Precision	0,94	0,94
Weighted Recall	0,94	0,94
Weighted F1-score	0,94	0,94

Chỉ số	Float32	INT8
AUC (hiển thị, làm tròn)	1,00	1,00

Trên bảng Confusion Matrix INT8, tỷ lệ nhận diện đúng lần lượt là Glioma 93,3%, Meningioma 84,2%, No Tumor 99,3% và Pituitary 99,0%. Meningioma là lớp có tỷ lệ nhận diện đúng thấp nhất; bảng cũng có cột Uncertain đối với mẫu chưa được xác nhận chắc chắn theo ngưỡng của hệ thống.

Step 13. Xuất mô hình đã huấn luyện từ Dashboard

Vào Dashboard → Download block output, chọn đúng Impulse #1. Tải ba định dạng: Keras h5 model, TensorFlow Lite (float32) và TensorFlow Lite (int8 quantized). Không nhầm file training data NPY với mô hình CNN.

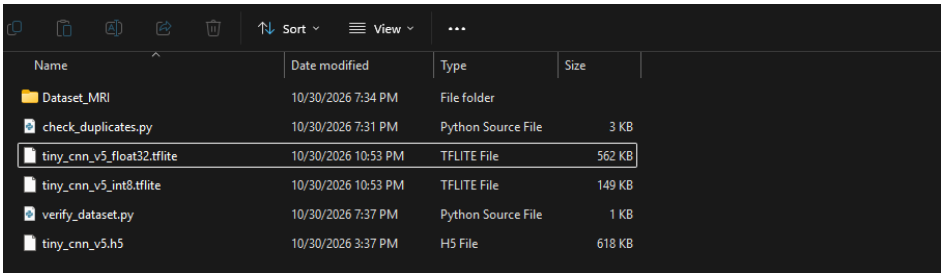


Hình 3.4. Các định dạng mô hình được cung cấp bởi Edge Impulse.

Step 14. Chuẩn hóa tên và lưu mô hình trên máy tính

Giải nén file ZIP Keras để lấy model.h5 rồi đổi tên thành tiny_cnn_v5.h5. Hai file .lite đổi phần mở rộng thành .tflite, giữ nguyên nội dung nhị phân. Đặt cả ba tệp trong thư mục D:\SoC_PYNQ_Z2_MRI_CNN\Source; giữ nguyên Dataset_MRI và các script kiểm tra ảnh đã có.

```
Source/
  Dataset_MRI/
  check_duplicates.py
  verify_dataset.py
  tiny_cnn_v5.h5
  tiny_cnn_v5_float32.tflite
  tiny_cnn_v5_int8.tflite
```



Hình 3.5. Ba file mô hình sau khi chuẩn hóa tên tại thư mục Source.

Step 15. Xác minh trọng số Conv2D phục vụ Lab 04

Kiểm tra trực tiếp file Keras H5 đã xuất. Input Layer có dạng 96 × 96 × 1; lớp conv1 có tensor kernel (3, 3, 1, 8) và bias (8,). Với padding valid, stride 1, lớp conv1 sinh feature map 94 × 94 × 8. Kết quả này là cấu hình đầu vào của bộ tăng tốc phần cứng dự kiến, chưa phải số liệu đo thực tế trên board.

Đối tượng	**Giá trị xác minh**
Input	(96, 96, 1)
conv1/kernel	(3, 3, 1, 8)
conv1/bias	(8,)
Output conv1	(94, 94, 8)
Số MAC của conv1	94 × 94 × 8 × 3 × 3 = 636.192

Trong Lab 04 sẽ trích xuất kernel và bias, tạo phép tính Conv2D tham chiếu trên CPU, kiểm tra sai số và thiết kế IP HLS. Chưa khẳng định hệ thống ARM + FPGA đã chạy hoặc tăng tốc ở thời điểm kết thúc Lab 03. Các trị số latency/tài nguyên trên giao diện Edge Impulse đang phụ thuộc target được chọn, không dùng thay cho phép đo PYNQ-Z2.

Phạm vi kiểm tra trùng dữ liệu từ Lab 02 là so sánh SHA-256 các file ảnh. Chưa có căn cứ xác nhận dữ liệu Train và Test hoàn toàn độc lập theo bệnh nhân; hạn chế này phải được ghi rõ khi diễn giải kết quả nghiên cứu.